

Python Module 07

Abstract Card Architecture

Complete beginner study guide + subject-based code

This guide follows the supplied Module 07 DataDeck subject: abstract factories, separate capabilities, multiple inheritance, and battle strategies.

Important: The subject says evaluation focuses on understanding the design patterns, not only the implementation. Use this as a study copy, then rebuild the important classes yourself without looking.

1. What the subject requires

- Python 3.10 or later.
- flake8 coding style.
- Comprehensive type annotations, checked with mypy.
- Built-in functions and standard classes/collections are allowed, except `eval()` and `exec()`.
- External libraries are forbidden.
- Functions should handle exceptions gracefully instead of crashing.
- Each exercise folder must contain `__init__.py` and be a package.
- The testing scripts `battle.py`, `capacitor.py`, and `tournament.py` stay at the repository root.
- The activity assumes you already know inheritance, abstract classes, polymorphism, and imports.
- Exercise 1 must build on Exercise 0; Exercise 2 uses the earlier factories and capabilities.

Authorized imports: Each exercise allows builtins, standard types, typing, and `abc`. This solution uses `abc` plus imports from the local exercise packages.

Implementation choice: The subject requires `ex0/`, `ex1/`, and `ex2/` as packages but does not prescribe all internal filenames. The file organization in this guide is a clean choice that keeps abstract bases, concrete classes, factories, capabilities, and strategies separated.

2. New subjects to learn first

Abstract class: A class used as a blueprint. You normally do not create it directly.

ABC: Python helper from `abc` that marks a class as an abstract base class.

@abstractmethod: Marks a method that concrete child classes must implement.

Concrete class: A finished class that implements all required abstract methods and can be instantiated.

Inheritance: A child class reuses and specializes behaviour from a parent class.

Method override: A child class provides its own version of a parent method, such as `attack()`.

Polymorphism: Different objects can be used through the same parent interface. The correct overridden method runs automatically.

Abstract Factory pattern: Code asks a factory for related objects instead of constructing concrete classes itself.

Capability interface: A small abstract class that describes one optional ability, such as `heal` or `transform`.

Multiple inheritance: One class inherits from more than one parent, for example `Creature` and `HealCapability`.

Persistent state: An attribute remembers a condition between method calls. `is_transformed` stays `True` until `revert()`.

Strategy pattern: Battle behaviour is placed in separate strategy objects so tournament code does not need many `if/elif` branches.

isinstance(): Checks whether an object supports a class/capability at runtime.

Custom exception: A project-specific exception class used to report one clear type of error.

Tuple: A fixed group of values. The tournament stores (factory, strategy) pairs.

Nested loop: A loop inside another loop. Here it creates every unique opponent pairing.

Study the design pattern, not only the final output.

`__all__`: Lists names intended to be public when a package is imported.

The big idea: Factories answer "what object should be created?" Capabilities answer "what extra ability does this object have?" Strategies answer "how should this object act in this situation?"

3. Project tree used in this guide

```
.
|-- battle.py
|-- capacitor.py
|-- tournament.py
|-- ex0
|   |-- __init__.py
|   |-- creature.py
|   |-- creatures.py
|   `-- factory.py
|-- ex1
|   |-- __init__.py
|   |-- capabilities.py
|   |-- creatures.py
|   `-- factories.py
`-- ex2
    |-- __init__.py
    `-- strategies.py
```

Package exposure rule: `ex0/__init__.py` exports factories only, not `Flameling`, `Pyrodon`, `Aquabub`, or `Torragon`. `ex1/__init__.py` also exports only its two factories. The concrete `Creature` classes stay internal to their packages.

4. Exercise 0 - Creature Factory

Goal: create families of related `Creature` objects through an abstract factory. `battle.py` should work with the common `CreatureFactory` type rather than knowing how each concrete `Creature` is constructed.

`ex0/creature.py`

What it does: Defines the `Creature` abstract base class. Every `Creature` has a name and type, must implement `attack()`, and inherits the ready-made `describe()` method.

New thing to learn: `ABC`, `@abstractmethod`, instance attributes, abstract versus concrete methods.

Beginner note: The abstract `attack()` raises `NotImplementedError` as a safety fallback, but the `@abstractmethod` decorator is what prevents incomplete `Creature` subclasses from being instantiated.

```
from abc import ABC, abstractmethod

class Creature(ABC):
    """Base class shared by every Creature card."""

    def __init__(self, name: str, creature_type: str) -> None:
        self.name: str = name
        self.creature_type: str = creature_type

    @abstractmethod
    def attack(self) -> str:
        """Return the Creature's attack message."""
        raise NotImplementedError

    def describe(self) -> str:
```

Study the design pattern, not only the final output.

```
"""Return a common description for every Creature."""
return f"{self.name} is a {self.creature_type} type Creature"
```

ex0/creatures.py

What it does: Implements Flameling, Pyrodon, Aquabub, and Torragon. Each class sets its own name/type and overrides attack().

New thing to learn: Inheritance, super(), overriding, and polymorphism.

Beginner note: All four classes share describe() from Creature. Only the data and attack() method change. This avoids copying the same description logic four times.

```
from .creature import Creature

class Flameling(Creature):
    def __init__(self) -> None:
        super().__init__("Flameling", "Fire")

    def attack(self) -> str:
        return "Flameling uses Ember!"

class Pyrodon(Creature):
    def __init__(self) -> None:
        super().__init__("Pyrodon", "Fire/Flying")

    def attack(self) -> str:
        return "Pyrodon uses Flamethrower!"

class Aquabub(Creature):
    def __init__(self) -> None:
        super().__init__("Aquabub", "Water")

    def attack(self) -> str:
        return "Aquabub uses Water Gun!"

class Torragon(Creature):
    def __init__(self) -> None:
        super().__init__("Torragon", "Water")

    def attack(self) -> str:
        return "Torragon uses Hydro Pump!"
```

ex0/factory.py

What it does: Defines CreatureFactory with create_base() and create_evolved(), then implements FlameFactory and AquaFactory.

New thing to learn: The Abstract Factory design pattern.

Beginner note: battle.py can accept CreatureFactory and does not need if family == "fire" or if family == "water". The factory decides which concrete classes to build.

```
from abc import ABC, abstractmethod

from .creature import Creature
from .creatures import Aquabub, Flameling, Pyrodon, Torragon

class CreatureFactory(ABC):
    """Interface for creating one Creature family."""
```

```

@abstractmethod
def create_base(self) -> Creature:
    """Create the base Creature of a family."""
    raise NotImplementedError

@abstractmethod
def create_evolved(self) -> Creature:
    """Create the evolved Creature of a family."""
    raise NotImplementedError

class FlameFactory(CreatureFactory):
    def create_base(self) -> Flameling:
        return Flameling()

    def create_evolved(self) -> Pyrodon:
        return Pyrodon()

class AquaFactory(CreatureFactory):
    def create_base(self) -> Aquabub:
        return Aquabub()

    def create_evolved(self) -> Torragon:
        return Torragon()

```

ex0/__init__.py

What it does: Defines the public ex0 package interface and exports factory classes only.

New thing to learn: __init__.py, __all__, and hiding concrete implementation details.

Beginner note: A user can write from ex0 import FlameFactory, but ex0 does not offer Flameling as a top-level public name.

```

from .factory import AquaFactory, CreatureFactory, FlameFactory

__all__ = ["AquaFactory", "CreatureFactory", "FlameFactory"]

```

battle.py

What it does: Creates the flame and aqua factories, tests base/evolved creation through one generic test_factory() function, then makes the two base Creatures fight.

New thing to learn: Programming to an abstract interface instead of concrete classes.

Beginner note: Notice that test_factory() receives CreatureFactory. The same function works for both families because both factories follow the same contract.

```

from ex0 import AquaFactory, CreatureFactory, FlameFactory

def test_factory(factory: CreatureFactory) -> None:
    print("Testing factory")
    base = factory.create_base()
    evolved = factory.create_evolved()
    print(base.describe())
    print(base.attack())
    print(evolved.describe())
    print(evolved.attack())

def battle(
    first_factory: CreatureFactory,
    second_factory: CreatureFactory,
) -> None:
    print("Testing battle")

```

Study the design pattern, not only the final output.

```

first = first_factory.create_base()
second = second_factory.create_base()
print(first.describe())
print("vs.")
print(second.describe())
print("fight!")
print(first.attack())
print(second.attack())

def main() -> None:
    flame_factory = FlameFactory()
    aqua_factory = AquaFactory()
    test_factory(flame_factory)
    test_factory(aqua_factory)
    battle(flame_factory, aqua_factory)

if __name__ == "__main__":
    main()

```

Exercise 0 mental model: CreatureFactory is a vending-machine interface. FlameFactory and AquaFactory are two different machines. You press "base" or "evolved" and each machine returns the correct Creature family without the caller knowing the constructor.

5. Exercise 1 - Capabilities

Goal: add optional abilities without putting every possible ability inside the Creature base class. Healing and transformation are separate abstract capabilities, then selected Creature classes inherit the ability they need.

[ex1/capabilities.py](#)

What it does: Defines HealCapability with heal(), and TransformCapability with transform(), revert(), and persistent is_transformed state.

New thing to learn: Small interfaces/capabilities and stateful behaviour.

Beginner note: The capability classes do not inherit Creature. That keeps healing and transformation reusable for other kinds of objects in the future.

```

from abc import ABC, abstractmethod

class HealCapability(ABC):
    """Interface for objects that know how to heal."""

    @abstractmethod
    def heal(self) -> str:
        """Return a healing action message."""
        raise NotImplementedError

class TransformCapability(ABC):
    """Interface for objects that can transform and revert."""

    def __init__(self) -> None:
        self.is_transformed: bool = False

    @abstractmethod
    def transform(self) -> str:
        """Enter the transformed state."""
        raise NotImplementedError

```

Study the design pattern, not only the final output.

```
@abstractmethod
def revert(self) -> str:
    """Return to the normal state."""
    raise NotImplementedError
```

ex1/creatures.py

What it does: Creates Sproutling/Bloomelle with healing and Shiftling/Morphagon with transformation. Transforming attacks change while `is_transformed` is `True`.

New thing to learn: Multiple inheritance and state-dependent method behaviour.

Beginner note: Shiftling inherits both Creature and TransformCapability. `transform()` changes the state, `attack()` reads that state, and `revert()` restores it.

```
from ex0.creature import Creature

from .capabilities import HealCapability, TransformCapability

class Sproutling(Creature, HealCapability):
    def __init__(self) -> None:
        Creature.__init__(self, "Sproutling", "Grass")

    def attack(self) -> str:
        return "Sproutling uses Vine Whip!"

    def heal(self) -> str:
        return "Sproutling heals itself for a small amount"

class Bloomelle(Creature, HealCapability):
    def __init__(self) -> None:
        Creature.__init__(self, "Bloomelle", "Grass/Fairy")

    def attack(self) -> str:
        return "Bloomelle uses Petal Dance!"

    def heal(self) -> str:
        return "Bloomelle heals itself and others for a large amount"

class Shiftling(Creature, TransformCapability):
    def __init__(self) -> None:
        Creature.__init__(self, "Shiftling", "Normal")
        TransformCapability.__init__(self)

    def attack(self) -> str:
        if self.is_transformed:
            return "Shiftling performs a boosted strike!"
        return "Shiftling attacks normally."

    def transform(self) -> str:
        self.is_transformed = True
        return "Shiftling shifts into a sharper form!"

    def revert(self) -> str:
        self.is_transformed = False
        return "Shiftling returns to normal."

class Morphagon(Creature, TransformCapability):
    def __init__(self) -> None:
        Creature.__init__(self, "Morphagon", "Normal/Dragon")
        TransformCapability.__init__(self)

    def attack(self) -> str:
        if self.is_transformed:
```

Study the design pattern, not only the final output.

```

        return "Morphagon unleashes a devastating morph strike!"
    return "Morphagon attacks normally."

    def transform(self) -> str:
        self.is_transformed = True
        return "Morphagon morphs into a dragonic battle form!"

    def revert(self) -> str:
        self.is_transformed = False
        return "Morphagon stabilizes its form."

```

ex1/factories.py

What it does: Builds the healing family and transforming family through the CreatureFactory interface introduced in Exercise 0.

New thing to learn: Extending an existing abstraction instead of rewriting it.

Beginner note: This is why Exercise 1 is built on Exercise 0. The new factories fit into the same `create_base()/create_evolved()` design.

```

from ex0.factory import CreatureFactory

from .creatures import Bloomelle, Morphagon, Shiftling, Sproutling

class HealingCreatureFactory(CreatureFactory):
    def create_base(self) -> Sproutling:
        return Sproutling()

    def create_evolved(self) -> Bloomelle:
        return Bloomelle()

class TransformCreatureFactory(CreatureFactory):
    def create_base(self) -> Shiftling:
        return Shiftling()

    def create_evolved(self) -> Morphagon:
        return Morphagon()

```

ex1/__init__.py

What it does: Exports only HealingCreatureFactory and TransformCreatureFactory.

New thing to learn: Keeping concrete classes private behind factories.

```

from .factories import HealingCreatureFactory, TransformCreatureFactory

__all__ = ["HealingCreatureFactory", "TransformCreatureFactory"]

```

capacitor.py

What it does: Tests healing on base/evolved Creatures and transformation on base/evolved Creatures, including the changed attack after `transform()`.

New thing to learn: Calling methods from multiple inherited interfaces and observing persistent state.

Beginner note: The order matters for transform Creatures: normal attack -> transform -> boosted attack -> revert. That sequence proves the state is persistent between calls.

```

from ex1 import HealingCreatureFactory, TransformCreatureFactory

def test_healing(factory: HealingCreatureFactory) -> None:

```

Study the design pattern, not only the final output.

```

print("Testing Creature with healing capability")
base = factory.create_base()
evolved = factory.create_evolved()

print("base:")
print(base.describe())
print(base.attack())
print(base.heal())

print("evolved:")
print(evolved.describe())
print(evolved.attack())
print(evolved.heal())

def test_transform(factory: TransformCreatureFactory) -> None:
    print("Testing Creature with transform capability")
    base = factory.create_base()
    evolved = factory.create_evolved()

    print("base:")
    print(base.describe())
    print(base.attack())
    print(base.transform())
    print(base.attack())
    print(base.revert())

    print("evolved:")
    print(evolved.describe())
    print(evolved.attack())
    print(evolved.transform())
    print(evolved.attack())
    print(evolved.revert())

def main() -> None:
    healing_factory = HealingCreatureFactory()
    transform_factory = TransformCreatureFactory()
    test_healing(healing_factory)
    test_transform(transform_factory)

if __name__ == "__main__":
    main()

```

Exercise 1 mental model: Creature says what every card must be able to do. A capability says what only some cards can additionally do. Multiple inheritance combines the required Creature contract with one optional ability.

6. Exercise 2 - Abstract Strategy

Goal: move battle behaviour out of tournament.py. The tournament receives a factory plus a strategy, and each strategy knows how to act with a suitable Creature.

[ex2/strategies.py](#)

What it does: Defines BattleStrategy, NormalStrategy, AggressiveStrategy, DefensiveStrategy, and the dedicated InvalidStrategyError.

New thing to learn: Strategy pattern, runtime capability checks, and custom exceptions.

Beginner note: Normal works for every Creature. Aggressive requires TransformCapability. Defensive requires HealCapability. isinstance() checks those capabilities without testing concrete names like "Shiftling" or "Sproutling".

```

from abc import ABC, abstractmethod

from ex0.creature import Creature
from ex1.capabilities import HealCapability, TransformCapability

class InvalidStrategyError(Exception):
    """Raised when a strategy cannot be used by a Creature."""

class BattleStrategy(ABC):
    """Interface used by the tournament to choose battle behaviour."""

    @abstractmethod
    def is_valid(self, creature: Creature) -> bool:
        """Return True when this strategy supports the Creature."""
        raise NotImplementedError

    @abstractmethod
    def act(self, creature: Creature) -> str:
        """Perform the strategy and return printable action lines."""
        raise NotImplementedError

class NormalStrategy(BattleStrategy):
    def is_valid(self, creature: Creature) -> bool:
        return True

    def act(self, creature: Creature) -> str:
        return creature.attack()

class AggressiveStrategy(BattleStrategy):
    def is_valid(self, creature: Creature) -> bool:
        return isinstance(creature, TransformCapability)

    def act(self, creature: Creature) -> str:
        if not isinstance(creature, TransformCapability):
            raise InvalidStrategyError(
                f"Invalid Creature '{creature.name}' for this "
                "aggressive strategy"
            )
        return "\n".join(
            [creature.transform(), creature.attack(), creature.revert()]
        )

class DefensiveStrategy(BattleStrategy):
    def is_valid(self, creature: Creature) -> bool:
        return isinstance(creature, HealCapability)

    def act(self, creature: Creature) -> str:
        if not isinstance(creature, HealCapability):
            raise InvalidStrategyError(
                f"Invalid Creature '{creature.name}' for this "
                "defensive strategy"
            )
        return "\n".join([creature.attack(), creature.heal()])

```

ex2/ __init__.py

What it does: Exports the strategy classes and the dedicated strategy error through ex2.

New thing to learn: A clean package interface for behaviour objects.

```

from .strategies import (
    AggressiveStrategy,
    BattleStrategy,
    DefensiveStrategy,

```

```

        InvalidStrategyError,
        NormalStrategy,
    )
    __all__ = [
        "AggressiveStrategy",
        "BattleStrategy",
        "DefensiveStrategy",
        "InvalidStrategyError",
        "NormalStrategy",
    ]

```

tournament.py

What it does: Builds three tournaments. battle() receives a list of (factory, strategy) tuples, makes every unique pair fight, validates strategies, and catches the dedicated exception to abort cleanly on an invalid combination.

New thing to learn: Tuples, lists, nested loops, all-vs-all pairing, is_valid(), try/except, and strategy objects.

Beginner note: The nested loops use second_index = first_index + 1 so A vs B happens once and B vs A is not repeated. With three opponents, the pairs are 1-2, 1-3, and 2-3.

```

from ex0 import AquaFactory, CreatureFactory, FlameFactory
from ex1 import HealingCreatureFactory, TransformCreatureFactory
from ex2 import (
    AggressiveStrategy,
    BattleStrategy,
    DefensiveStrategy,
    InvalidStrategyError,
    NormalStrategy,
)

Opponent = tuple[CreatureFactory, BattleStrategy]

def battle(opponents: list[Opponent]) -> None:
    print("*** Tournament ***")
    print(f"{len(opponents)} opponents involved")

    for first_index in range(len(opponents)):
        for second_index in range(first_index + 1, len(opponents)):
            first_factory, first_strategy = opponents[first_index]
            second_factory, second_strategy = opponents[second_index]
            first = first_factory.create_base()
            second = second_factory.create_base()

            print("* Battle *")
            print(first.describe())
            print("vs.")
            print(second.describe())
            print("now fight!")

            try:
                if not first_strategy.is_valid(first):
                    first_strategy.act(first)
                if not second_strategy.is_valid(second):
                    second_strategy.act(second)
                print(first_strategy.act(first))
                print(second_strategy.act(second))
            except InvalidStrategyError as error:
                print(f"Battle error, aborting tournament: {error}")
                return

def main() -> None:

```

```

normal = NormalStrategy()
aggressive = AggressiveStrategy()
defensive = DefensiveStrategy()

print("Tournament 0 (basic)")
print("[ (Flameling+Normal), (Healing+Defensive) ]")
battle(
    [
        (FlameFactory(), normal),
        (HealingCreatureFactory(), defensive),
    ]
)

print("Tournament 1 (error)")
print("[ (Flameling+Aggressive), (Healing+Defensive) ]")
battle(
    [
        (FlameFactory(), aggressive),
        (HealingCreatureFactory(), defensive),
    ]
)

print("Tournament 2 (multiple)")
print(
    "[ (Aquabub+Normal), (Healing+Defensive), "
    "(Transform+Aggressive) ]"
)
battle(
    [
        (AquaFactory(), normal),
        (HealingCreatureFactory(), defensive),
        (TransformCreatureFactory(), aggressive),
    ]
)

if __name__ == "__main__":
    main()

```

Why strategy is useful: Without strategies, tournament.py would need logic such as "if healing, attack then heal; elif transforming, transform then attack then revert". Strategy objects remove those family-specific branches from the tournament.

7. How the three patterns connect

Factory: Creates the right Creature family. Example: FlameFactory creates Flameling/Pyrodon.

Capability: Adds an optional ability to selected Creature classes. Example: TransformCapability gives transform() and revert().

Strategy: Chooses a battle algorithm independently of the Creature constructor. Example: AggressiveStrategy performs transform -> attack -> revert.

One-sentence evaluation answer: The project separates object creation (factory), optional object abilities (capabilities), and situational behaviour (strategy), so each part can change without forcing a rewrite of the others.

8. Expected outputs from this implementation

The implementation was executed with Python and produces the subject example behaviour.

battle.py

```
Testing factory
Flameling is a Fire type Creature
Flameling uses Ember!
Pyrodon is a Fire/Flying type Creature
Pyrodon uses Flamethrower!
Testing factory
Aquabub is a Water type Creature
Aquabub uses Water Gun!
Torrragon is a Water type Creature
Torrragon uses Hydro Pump!
Testing battle
Flameling is a Fire type Creature
vs.
Aquabub is a Water type Creature
fight!
Flameling uses Ember!
Aquabub uses Water Gun!
```

capacitor.py

```
Testing Creature with healing capability
base:
Sproutling is a Grass type Creature
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount
evolved:
Bloomelle is a Grass/Fairy type Creature
Bloomelle uses Petal Dance!
Bloomelle heals itself and others for a large amount
Testing Creature with transform capability
base:
Shiftling is a Normal type Creature
Shiftling attacks normally.
Shiftling shifts into a sharper form!
Shiftling performs a boosted strike!
Shiftling returns to normal.
evolved:
Morphagon is a Normal/Dragon type Creature
Morphagon attacks normally.
Morphagon morphs into a dragonic battle form!
Morphagon unleashes a devastating morph strike!
Morphagon stabilizes its form.
```

tournament.py

```
Tournament 0 (basic)
[ (Flameling+Normal), (Healing+Defensive) ]
*** Tournament ***
2 opponents involved
* Battle *
Flameling is a Fire type Creature
vs.
Sproutling is a Grass type Creature
now fight!
Flameling uses Ember!
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount
Tournament 1 (error)
[ (Flameling+Aggressive), (Healing+Defensive) ]
*** Tournament ***
2 opponents involved
* Battle *
Flameling is a Fire type Creature
vs.
Sproutling is a Grass type Creature
now fight!
```

```

Battle error, aborting tournament: Invalid Creature 'Flameling' for this aggressive strategy
Tournament 2 (multiple)
[ (Aquabub+Normal), (Healing+Defensive), (Transform+Aggressive) ]
*** Tournament ***
3 opponents involved
* Battle *
Aquabub is a Water type Creature
vs.
Sproutling is a Grass type Creature
now fight!
Aquabub uses Water Gun!
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount
* Battle *
Aquabub is a Water type Creature
vs.
Shiftling is a Normal type Creature
now fight!
Aquabub uses Water Gun!
Shiftling shifts into a sharper form!
Shiftling performs a boosted strike!
Shiftling returns to normal.
* Battle *
Sproutling is a Grass type Creature
vs.
Shiftling is a Normal type Creature
now fight!
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount
Shiftling shifts into a sharper form!
Shiftling performs a boosted strike!
Shiftling returns to normal.

```

9. How to run and test the project

Run these from the repository root:

```

python3 battle.py
python3 capacitor.py
python3 tournament.py

```

Required quality checks

```

python3 -m compileall .
flake8 .
mypy .

```

What was verified here: The code in this guide was syntax-compiled and all three root scripts were run. Their important output matches the supplied examples. flake8 and mypy are required by the subject, so run those commands in your 42 environment as the final style/type checks.

Graceful error handling: Tournament 1 deliberately pairs Flameling with AggressiveStrategy. `is_valid()` rejects the combination, `act()` raises `InvalidStrategyError`, and `tournament.py` catches it and prints a clear error instead of exposing a traceback.

10. Questions you should be able to answer

Q: Why is Creature abstract?

A: It defines the common contract but leaves `attack()` to each concrete Creature.

Q: What is the difference between an abstract and concrete method?

Study the design pattern, not only the final output.

A: An abstract method must be implemented by a child class. A concrete method already has usable code, such as `describe()`.

Q: What problem does Abstract Factory solve here?

A: It creates related base/evolved objects without the caller depending on concrete Creature constructors.

Q: Why does battle.py accept CreatureFactory?

A: Polymorphism lets one function work with FlameFactory, AquaFactory, and later factory subclasses.

Q: Why are capability classes separate from Creature?

A: Healing or transformation might later apply to other object types, and not every Creature needs every capability.

Q: What is multiple inheritance doing?

A: Sproutling combines Creature plus HealCapability; Shiftling combines Creature plus TransformCapability.

Q: Why is `is_transformed` an attribute?

A: It remembers state between `transform()`, `attack()`, and `revert()` calls.

Q: What is a strategy object?

A: An object that contains one algorithm for how a Creature should act during battle.

Q: Why does AggressiveStrategy use `isinstance()`?

A: It checks for TransformCapability instead of hard-coding concrete Creature class names.

Q: Why create InvalidStrategyError?

A: It distinguishes a strategy/Creature mismatch from unrelated errors and lets `tournament.py` handle it clearly.

Q: Why use a list of tuples in `tournament.py`?

A: Each tuple keeps one CreatureFactory paired with its chosen BattleStrategy.

Q: Why do the nested loops start the second index after the first?

A: It generates every unique pair exactly once and avoids self-battles and duplicate reverse battles.

Q: Factory vs Strategy?

A: Factory decides what object to create. Strategy decides what behaviour to execute.

11. Beginner mistakes to avoid

- Instantiating Creature or CreatureFactory directly. They are abstract blueprints.
- Forgetting `@abstractmethod` or forgetting to implement an abstract method in a concrete child.
- Importing concrete Creature classes from `ex0/__init__.py` or `ex1/__init__.py` and breaking the factory-only public interface.
- Putting `heal()`, `transform()`, and `revert()` directly in Creature even though they are optional capabilities.

- Forgetting to call `TransformCapability.__init__()`, leaving `is_transformed` undefined.
- Changing transformed state but making `attack()` ignore the state.
- Checking concrete names inside strategies instead of checking the required capability.
- Calling an invalid strategy without raising the dedicated exception.
- Catching every Exception broadly when you only need to catch `InvalidStrategyError`.
- Writing tournament battle logic with many if/elif family checks instead of calling `strategy.act()`.
- Using nested loops that create A vs A or repeat both A vs B and B vs A.
- Forgetting `__init__.py` in `ex0/`, `ex1/`, or `ex2/`.
- Submitting code that runs but cannot be explained during evaluation.

12. Beginner submission checklist

- Confirm `battle.py` is at the repository root and `ex0/` is a package.
- Confirm `capacitor.py` is at the repository root and `ex1/` builds on `ex0`.
- Confirm `tournament.py` is at the repository root and `ex2/` uses earlier factories/capabilities.
- Confirm every exercise folder contains `__init__.py`.
- Confirm `ex0` and `ex1` top-level package interfaces expose factories, not concrete Creature classes.
- Run `battle.py` and compare the Creature descriptions/attacks with the subject example.
- Run `capacitor.py` and verify transform changes attack behaviour before `revert()`.
- Run `tournament.py` and verify the invalid `Flameling+Aggressive` pairing is handled with a clear message.
- Run `python3 -m compileall .`, `flake8 .`, and `mypy .`
- Remove external-library imports; none are needed.
- Practice drawing the inheritance relationships on paper.
- Practice explaining Abstract Factory, capability interfaces, multiple inheritance, state, `Strategy`, `isinstance()`, and custom exceptions.
- Commit the required files to Git and double-check filenames before evaluation.

Fast study order: 1) Understand Creature and CreatureFactory. 2) Trace one `FlameFactory` call from factory to `Flameling`. 3) Trace `Shiftling` state through `transform/attack/revert`. 4) Trace `AggressiveStrategy` using `TransformCapability`. 5) Draw the three tournament pairs created from three opponents.

Source basis: the user-supplied "DataDeck - Abstract Card Architecture", Module 07, Version 3.0. Requirements and example behaviour are kept aligned with that subject.